Deep Reinforcement Learning Project

Chris McKie

CSCI 264: Artificial Intelligence

1 Project Setup

The project was completed using Stable-Baselines3 to train agents to play two separate Atarti games[1]. All training was done on a MacBook Pro, containing an M2 Pro chip with a 10-core CPU and a 16 core GPU. Several agents were trained across different domains, algorithms, and learning rates, and each agent was trained in a separate Jupyter notebook. Evaluation and video generation were done in separate Python scripts.

2 Algorithms & Domain

2.1 Algorithms Used

Two algorithms were used for this project: Deep Q-Network and Proximal Policy Optimization.

The Deep Q-Network (DQN) algorithm is used as the primary baseline algorithm. It is sample based and works by learning a Q-function, which estimates the expected future reward for each action given the current state. At each new state, the agent picks the action with the highest predicted Q-value. DQN is an off-policy algorithm, so it learns from past experiences and converges to the optimal policy.

The Proximal Policy Optimization (PPO) algorithm that, instead of a Q-value, learns a policy, which is a probability distribution over possible actions given the current state. The PPO algorithm also learns a value function, which estimates how good any given state is. Unlike DQN, PPO is an on-policy algorithm, so it only learns from the current policy, rather than past experiences.

2.2 Chosen Domains

The chosen algorithms to run the algorithms on are Q-bert and Donkey Kong.

Q-bert is a puzzle game containing colored tiles, where stepping on a tile changes its color. The goal is for the player to jump on to each tile to change all the colors while avoiding enemies. Q-bert was chosen as a domain due to its repetitive gameplay making it well-sutied for an agent to learn patterns based on changing the tile colors.

Donkey Kong is a platformer game, where the player starts at the bottom of the level, and the goal is to reach the top. The player must jump over barrels, climb ladders, and jump on to moving platforms to reach the goal. Donkey Kong was chosen more as an experiment; its gameplay is less repetitive and more complicated than Q-bert, and the rewards are harder to achieve with more steps.

3 Hyperparameter Variations

Throughout the tests, the only hyperparameter that changed was the learning rate. This was done to isolate the learning rate and observe how it affected the agent across different domains and algorithms. The other hyperparameters were fixed depending on the algorithm, specified below.

3.1 DQN Parameters

- **Learning Rate α:** 1e-4 and 5e-5
- **Buffer Size:** 100,000 (Number of past experiences stored)
- **Learning Starts:** 10,000 (Number of steps before training begins)
- **Batch Size:** 32 (Number of random samples from the buffer in each gradient update)
- **Training Frequency:** 4 (Number of steps between gradient updates)
- **Target Update Interval:** 1,000 (Number of steps target weights are updated)

3.2 PPO Parameters

- **Learning Rate α:** 3e-4 and 1e-4
- n **Steps:** 128 (Number of steps between updates)
- **Batch size:** 256 (Number of steps per gradient update pass)
- n **Epochs:** 4 (How many times the collected batch is reused for gradient updates before discarded)
- **Entropy Coefficient:** 0.01 (Weight of entropy bonus to encourage exploration)
- **Clip Range:** 0.1 (Limit on how much the policy can change per update)
- n **Environments:** 8 (Number of parallel game instances running at the same time)

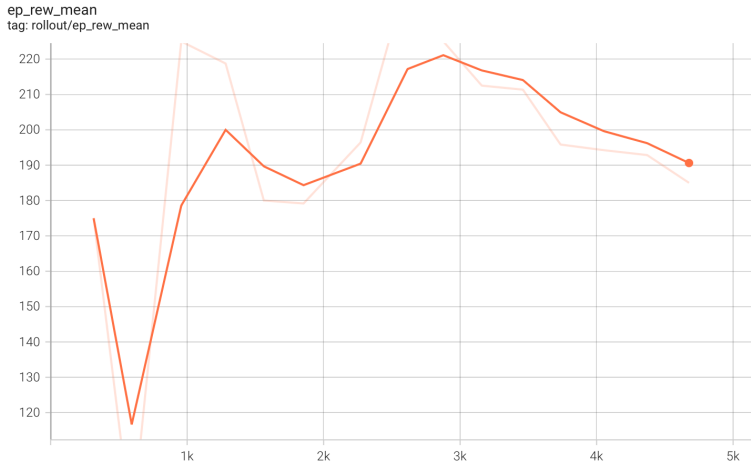

Figure 1: TensorBoard Learning Curve demonstrating restarting training from saved checkpoints. Training stopped at 500,000 steps (approximately 2,000 episodes in), with no effect on the learning curve.

4 Checkpoint & Resumed-Training Evidence

Checkpoints were saved every 100,000 steps of each training session. Q-bert training ran for 1 million steps while Donkey Kong ran for 2 million steps (longer than Q-bert to account for more complex gameplay), so 10 and 20 checkpoints were saved respectively.

The resumed-training evidence was demonstrated using the DQN Q-bert session (learning rate = 1e-4). Training was stopped at 500,000 steps and resumed to continue training. A learning curve created with TensorBoard, shown in Figure 1, shows Q-bert training with 200-250 steps per episode (approximally 4,600 episodes = 1 million steps). No reset on the learning curve confirms that the step count was successfully preservered upon restart.

5 Evaluation Summary

Q-bert was tested with four training sessions: Each algorithm was run twice, each using a different learning rate. Donkey Kong was tested with only two training sessions, with one run per algorithm, due to the longer training sessions each taking at least twice as long as the Q-bert runs.

Training results were recorded into a csv file, where the data was plotted onto charts and tables for evaluation. Table 1 shows the results after training based on the domain, algorithm, and learning rate. The learning curve of all six agents is compared directly in Figure 2. The mean rewards achieved by each model is shown in Figure 3. It should be noted that while both Donkey Kong and Q-bert are on the same graph, each game uses a different scoring system, so performance between each game is not directly comparable.

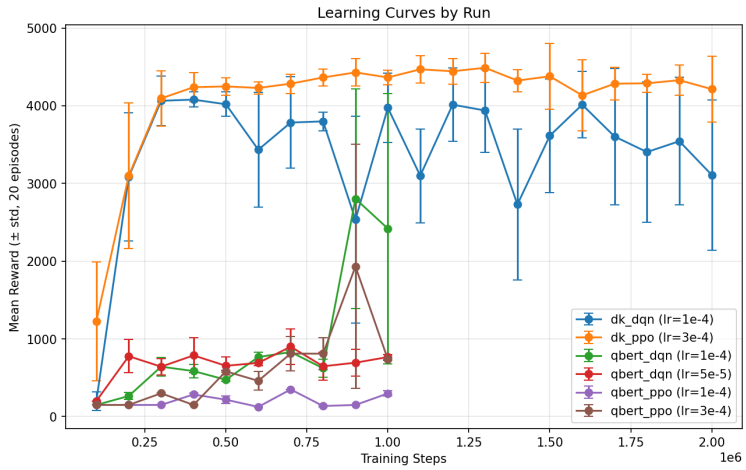

Figure 2: Learning curve of each agent. Note that Donkey Kong trained twice as long as Q-bert, which is why Q-bert learning curves end sooner.

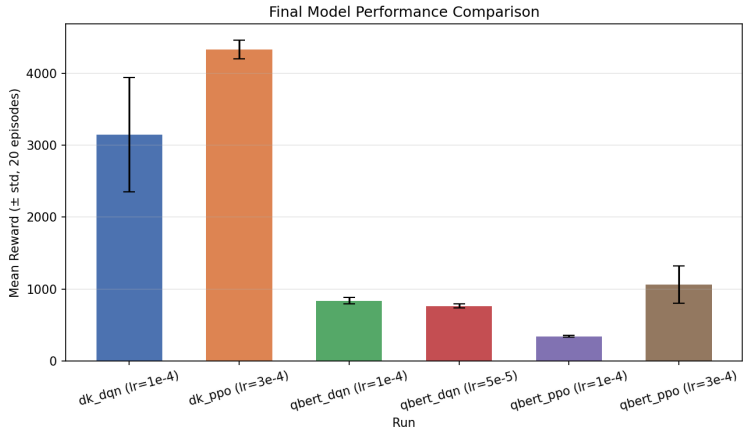

Figure 3: Bar chart showing final model performance. Note that Donkey Kong and Q-bert cannot be compared directly as they have different scoring systems.

Domain	Algorithm	Learning Rate	Mean Reward	Standard Deviation
Q-bert	DQN	1e-4	938.8	±47.7
Q-bert	DQN	5e-5	762.5	±29.0
Q-bert	PPO	3e-4	1062.5	±259.0
Q-bert	PPO	1e-4	342.5	±11.5
Donkey Kong	DQN	1e-4	3145.0	±797.2
Donkey Kong	PPO	3e-4	4330.0	±130.8

Table 1: Model performance showing mean reward and standard deviation.

6 Analysis & Interpretation of Results

The graphs and tables show some general trends and observations about performance across domains, algorithms, and learning rates.

6.1 General Observations

- Lower learning rates generally led to worse performance, and higher learning rates led to better performance.
- PPO generally outperformed DQN in both Q-Bert and Donkey Kong.
- Donkey Kong agent learned to prioritize jumping over barrels over completing levels, leading to the learning curve stabilizing quickly.
- Q-bert was slower to progress, but appeared to make drastic improvements towards the end.

6.2 Analysis Questions

6.2.1 Which algorithm performed best on each environment?

PPO outperformed DQN in both Q-bert and Donkey Kong. Donkey Kong was only tested once per algorithm, where PPO performed best. Q-bert was tested twice per algorithm, and the best performance used PPO.

6.2.2 How did the learning rate affect training?

Generally, training was better with higher learning rates. PPO appears to be more sensitive to learning rates than DQN, as reducing PPO's learning rate from 3e-4 to 1e-4 caused the mean reward to drop by 67%. Meanwhile, DQN did have a drop in the mean reward, but it was significantly less drastic than PPO.

6.2.3 Were the best quantitative models also the most convincing qualitatively in videos?

The best quantitative models produced mixed results in the videos. For Q-bert, the final models visibly performed better than the early and middle videos, which is in line with the higher scores.

The Donkey Kong models produced unexpected but interesting results: Instead of qualitatively improving in performance by completing levels, the model learned to exploit jumping over barrels for points early in the game rather than climb ladders and progress through the levels. This resulted in high quantitative scores, but low performance quality.

6.2.4 How sensitive were results to early training instability?

The Donkey Kong models converged to the exploitative barrel jumping policy fairly quickly, then remained fairly stable, although the PPO algorithm was much more stable than the DQN algorithm.

The Q-bert agents were much slower to learn, with all four models trending towards improvements towards the end of their training, suggesting the models had not fully converged, and longer training may have led to higher scores.

References

[1] C. McKie, "Deep reinforcement learning stable-baselines3 atari mini project," https://github.com/chrismckie/RL-Atari-Project, gitHub.